微信小游戏完整流程

Canvas 三消示例 · 纯广告 IAA · 无内购。配套：02 广告模板、03 验收表、04 备案文案、05 真机手册。

总顺序

1. 工具体系：Node、微信开发者工具、写代码工具（Cursor / VS Code / 网页 AI 三选一）
2. 公众平台注册小游戏，AppID；名称 = 备案名 = 游戏内标题
3. 本地工程 → `game.js` → 开发者工具能玩
4. 三消逻辑 + 关卡 + 存档
5. 广告代码 + 占位 adunit；开通流量主后换真 ID，真机验收
6. 隐私与用户相关配置（否则审不过、也留不住人）
7. 备案
8. 开发者工具上传 → 提审 → 发布
9. 拉新、攒 UV 开流量主、看留存再调广告

一、工具体系

1.1 必装

软件	干什么	检查
Node.js LTS	`npm install`、`npm run build` 生成 `game.js`	终端 `node -v` 有版本号
微信开发者工具	编译预览、上传代码、体验版二维码、真机调试	扫码登录，能导入项目

下载：Node 官网；微信开发者工具见开放文档 · 开发者工具。

1.2 写代码（选一个）

工具	用法
Cursor	打开项目文件夹，中文描述需求，直接改 `src/`
VS Code + Copilot 等插件	同 Cursor
ChatGPT / Claude / 国内大模型网页	贴报错和代码，改完自己保存到项目

AI 只帮你写代码，**不能**代替：注册 AppID、开通流量主、点上传/提审。不要向 AI 粘贴真实 AppID、`adunit-`、云 envId。

不必为本路线安装 Cocos / Unity。

1.3 每天固定流程

```
改 src 或 game.js → npm run build（或 npm run dev 监听）
→ 微信开发者工具点「编译」
→ 测玩法
→ 测广告：预览 → 手机扫码（必须真机）
```

提示词（可复制给 AI）：

```
微信小游戏，竖屏 Canvas，不用 Cocos。打包输出根目录 game.js。
8×8 三消，非法交换不扣步。
激励复活、结算插屏、大厅原生横幅；wx 广告只在一个 adapter 文件里。
先目录结构，再最小可运行代码。
```

二、注册小游戏

1. 微信公众平台 → 注册 → 小游戏
2. 记下 **AppID**、小游戏名称（与备案、游戏内标题一致）
3. 设置 → 成员管理 → 加开发者微信号
4. 按类目上传要求准备材料（纯广告一般无内购；软著等以页面为准）
5. 广告入口：**流量主**（在公众平台内，不接第三方 SDK）

测广告必须用**正式 AppID**，游客/测试号能力不全。

三、创建小游戏（工程 + 玩法）

3.1 目录建议

```
config/           关卡、广告 ID
logic/            玩法，不写 wx.*
core/adapters/    唯一调用 wx 广告/存储的地方
presentation/     绘制、按钮
```

3.2 最小三文件工程

```
my-game/
├── game.json
├── project.config.json
└── game.js
```

`game.json`：

```
{ "deviceOrientation": "portrait", "showStatusBar": false }
```

`project.config.json`（改 `appid`）：

```
{
  "compileType": "game",
  "appid": "你的AppID",
  "projectname": "my-game",
  "packOptions": {
    "ignore": [
      { "value": "node_modules", "type": "folder" },
      { "value": "src", "type": "folder" },
      { "value": "scripts", "type": "folder" },
      { "value": "package.json", "type": "file" },
      { "value": "package-lock.json", "type": "file" },
      { "value": "game.js.map", "type": "file" }
    ]
  }
}
```

`game.js` 最小验证（能画、能点）：

```
const canvas = wx.createCanvas();
const ctx = canvas.getContext('2d');
const info = wx.getSystemInfoSync();
canvas.width = info.windowWidth * info.pixelRatio;
canvas.height = info.windowHeight * info.pixelRatio;
ctx.scale(info.pixelRatio, info.pixelRatio);
let tapX = -1, tapY = -1;
wx.onTouchStart((e) => {
  tapX = e.touches[0].clientX;
  tapY = e.touches[0].clientY;
});
function frame() {
  ctx.fillStyle = '#ffe9d6';
  ctx.fillRect(0, 0, info.windowWidth, info.windowHeight);
  ctx.fillStyle = '#ff7a45';
  ctx.font = '24px sans-serif';
  ctx.fillText('Hello 小游戏', 40, 120);
  if (tapX >= 0) {
    ctx.beginPath();
    ctx.arc(tapX, tapY, 20, 0, Math.PI * 2);
    ctx.fill();
  }
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

开发者工具：**小游戏 → 导入项目 → 选目录 → 编译**。

3.3 TypeScript 打包

```
npm init -y && npm i -D esbuild typescript
```

`package.json`：`"type": "module"`，`scripts.build` / `scripts.dev` 指向 `node scripts/build.mjs`。

`scripts/build.mjs`：

```
import * as esbuild from 'esbuild';
const watch = process.argv.includes('--watch');
const options = {
  entryPoints: ['src/main.ts'],
  bundle: true,
  outfile: 'game.js',
  format: 'iife',
  platform: 'neutral',
  target: ['es2018'],
  sourcemap: true,
};
if (watch) {
  const ctx = await esbuild.context(options);
  await ctx.watch();
} else {
  await esbuild.build(options);
}
```

`wx` 类型：装小游戏类型声明，或 `declare const wx: any;`。

3.4 三消核心（8×8）

```
const ROWS = 8, COLS = 8, KINDS = 5, EMPTY = 0;
const randomKind = () => 1 + Math.floor(Math.random() * KINDS);

function createBoard() {
  const board = [];
  for (let r = 0; r < ROWS; r++) {
    board.push([]);
    for (let c = 0; c < COLS; c++) {
      let kind;
      do {
        kind = randomKind();
      } while (
        (c >= 2 && board[r][c - 1] === kind && board[r][c - 2] === kind) ||
        (r >= 2 && board[r - 1][c] === kind && board[r - 2][c] === kind)
      );
      board[r].push(kind);
    }
  }
  return board;
}

function findMatches(board) {
  const hit = new Set();
  for (let r = 0; r < ROWS; r++) {
    let run = 1;
    for (let c = 1; c <= COLS; c++) {
      const same = c < COLS && board[r][c] !== EMPTY && board[r][c] === board[r][c -
1];
      if (same) run++;
      else {
        if (run >= 3) for (let k = c - run; k < c; k++) hit.add(r * COLS + k);
        run = 1;
      }
    }
  }
  for (let c = 0; c < COLS; c++) {
    let run = 1;
    for (let r = 1; r <= ROWS; r++) {
      const same = r < ROWS && board[r][c] !== EMPTY && board[r][c] === board[r - 1]
[c];
      if (same) run++;
      else {
```

```
        if (run >= 3) for (let k = r - run; k < r; k++) hit.add(k * COLS + c);
        run = 1;
      }
    }
  }
  return hit;
}

function collapse(board) {
  for (let c = 0; c < COLS; c++) {
    let write = ROWS - 1;
    for (let r = ROWS - 1; r >= 0; r--) {
      if (board[r][c] !== EMPTY) {
        board[write][c] = board[r][c];
        if (write !== r) board[r][c] = EMPTY;
        write--;
      }
    }
    for (let r = write; r >= 0; r--) board[r][c] = randomKind();
  }
}

function resolve(board) {
  let cleared = 0;
  for (;;) {
    const hit = findMatches(board);
    if (hit.size === 0) return cleared;
    hit.forEach((i) => { board[Math.floor(i / COLS)][i % COLS] = EMPTY; });
    cleared += hit.size;
    collapse(board);
  }
}

const state = { board: createBoard(), movesLeft: 20, score: 0 };

function trySwap(r1, c1, r2, c2) {
  if (Math.abs(r1 - r2) + Math.abs(c1 - c2) !== 1) return false;
  const b = state.board;
  [b[r1][c1], b[r2][c2]] = [b[r2][c2], b[r1][c1]];
  if (findMatches(b).size === 0) {
    [b[r1][c1], b[r2][c2]] = [b[r2][c2], b[r1][c1]];
    return false;
  }
  state.movesLeft--;
```

```
  state.score += resolve(b) * 10;
  return true;
}
```

触摸转格子（`info` = `wx.getSystemInfoSync()`）：

```
const cell = Math.floor(Math.min(info.windowWidth, info.windowHeight) * 0.9 / COLS);
const boardLeft = (info.windowWidth - cell * COLS) / 2;
const boardTop = 160;
function hitCell(x, y) {
  const c = Math.floor((x - boardLeft) / cell);
  const r = Math.floor((y - boardTop) / cell);
  return r >= 0 && r < ROWS && c >= 0 && c < COLS ? { r, c } : null;
}
```

关卡与存档：

```
const LEVELS = [
  { id: 1, moves: 15, targetScore: 600 },
  { id: 2, moves: 15, targetScore: 900 },
];
function saveProgress(maxLevel) {
  wx.setStorageSync('progress', { maxLevel });
}
function loadProgress() {
  return wx.getStorageSync('progress') || { maxLevel: 1 };
}
```

首版顺序：能消 → 过关 → 存档 → 动画音效 → 接广告（第五节）。

3.5 广告位（写代码时预留）

类型	场景
激励	失败 +5 步；道具空；奖励关 +5 秒
插屏	通关结算；进大厅延迟
原生模板	仅大厅底，进关 hide

详细配置见 **02 广告配置模板.json**。

四、上传代码

上传的是**开发者工具打开的那个目录**（含 `game.js`、`game.json`、资源），不是 Git 仓库整包。

4.1 上传前

- [ ] `npm run build` 已执行，根目录 `game.js` 最新
- [ ] `packOptions.ignore` 已排除 `node_modules`、`src`、`game.js.map`
- [ ] 详情 → 本地代码：看主包大小是否超限
- [ ] 版本号心里有个数（上传时要填）

4.2 开发者工具里上传

1. 顶部菜单 **上传**（或工具栏上传按钮）
2. 填 **版本号**（如 1.0.0）和 **项目备注**（如：首版三消+广告占位）
3. 等待上传完成

4.3 公众平台侧

1. 登录 mp.weixin.qq.com → **管理** → **版本管理**
2. 看到刚上传的 **开发版**
3. 可先设 **体验版**（给测试扫码），确认玩法和广告后再提审
4. **提交审核** → 填审核说明（玩法路径、广告用途）
5. 审核通过 → **全量发布**（或灰度）

4.4 体验版给测试用户

版本管理 → 选开发版为体验版 → 扫体验版二维码。测广告、测存档、测弱网，**不要只用模拟器**。

4.5 常见上传问题

现象	处理
上传失败体积过大	检查 ignore；大图分包或 CDN
上传后白屏	线上包缺 `game.js` 或路径错
和本地不一致	忘 build 就上传

五、广告（流量主）

5.1 开通条件

流量主在公众平台申请，常见门槛：**有一定访问用户（UV）**、类目合规。很多新游戏要**先发布、先有人玩**，才能开。因此：

- 开发期：`adunit-YOUR_xxx` 占位，代码走通
- 上线后：达标 → 建广告位 → 换 ID → 再 upload 一版

5.2 建位与配置

流量主后台创建：激励 1～2、插屏 1、原生模板 1。ID 填入 **02 广告配置模板.json**（复制进项目）。

要点：

- `wx.createRewardedVideoAd` / `createInterstitialAd` / `createCustomAd`
- 每 placement 单例；`show` 只有 `completed` 发奖
- 激励：`isEnded` 或观看 ≥14s（与后台「最长激励时长」对齐）
- 插屏：每 2 关 + 60s 冷却，失败局不弹；启动插屏延迟 2～3s，2001 重试
- 横幅：仅大厅；进关 hide + onLoad 再检查

真机验收用 **03 验收表**。报错查 **05 手册**。

六、用户：审得过、玩得下、能变现

「用户」在这里分三块：**合规用户数据**、**拉人来玩**、**留存与广告平衡**。

6.1 隐私与用户授权（必做）

1. 公众平台 → 设置 → 用户隐私保护指引 → 填写并发布
2. `game.json` 按当前基础库要求配置隐私相关项（以微信文档为准）
3. 首次需要存储/相册等能力时，用官方隐私弹窗流程，**不要偷偷采集**

审核常因隐私指引缺失或类目与功能不符被打回。

6.2 让用户能进来（拉新）

手段	做法
搜索	名称、简介、关键词在公众平台填好；图标清晰
分享	`wx.shareAppMessage`，分享标题 + 图片（分享图在公众平台配置）
体验版	发给朋友扫码试玩，修 bug 再提审
游戏圈	可选，大厅入口；需按文档接按钮组件
视频号/公众号	有资源再导，与小游戏 AppID 同主体更方便

首版至少做：**分享** + **体验版找人试**。

分享代码示意：

```
wx.onShareAppMessage(() => ({
  title: '来消一局',
  imageUrl: 'images/share.jpg', // 路径按你项目资源
}));
```

6.3 让用户玩得下去（留存）

- 前 3 关难度偏低，别一上来卡死
- 插屏别过密（参考每 2 关 + 冷却）；新用户前几分钟少弹启动插屏
- 失败给 **重开**，激励复活是可选
- 关广告后别黑屏、别误触（见 05 手册）

6.4 用户量与流量主

- 没 UV → 流量主可能开不了 → 先靠分享、熟人体验版攒访问
- 流量主后台看曝光；1004 无填充时属正常时段/库存，提示用户即可
- 看 **次日留存**：留存掉还加广告 → 减插屏

6.5 审核里和「用户」有关的说明

提交审核时可写：

> 无内购。激励用于自愿复活/道具；插屏在通关结算；可分享邀请好友体验。隐私指引已配置。不看广告可继续重试或返回大厅。

七、备案

1. 按微信备案流程提交
2. 备案名 = 公众平台名 = 游戏内标题
3. 纯 IAA：系统描述不写商城/VIP/虚拟货币
4. 场景 / 玩法 / 系统截图各一套，**不要同一张图复用**
5. 文案用 **04 备案文案模板**，改【】

八、提审发布（ checklist ）

与 **03 验收表** 一起用。核心：

- 正式 AppID、无 `YOUR_` 广告占位（若已开流量主）
- 广告真机过
- 图标、简介、分享图、隐私指引
- 备案完成或平台允许的状态

通过后发布 → 正式版扫码再测广告和存档。

九、上线后

事项	动作
流量主	看 UV 是否达标，达标后换 adunit 再发版
收益	流量主后台；异常查广告位状态
留存	掉则减插屏、降关卡难度
用户反馈	真机复现；广告类先查 05

十、现象对照

现象	处理
白屏	build + 看真机日志
改代码无效	build + 重新编译
工具广告报错	真机
看完不发奖	第五节 激励发奖（isEnded + 观看时长）
启动插屏不出	2001 延迟重试
没广告	流量主/AppID/真机
审广告	去诱导
审名称	三处名称统一
审隐私	补隐私指引

规则以微信官方最新文档为准。